

AI Assistance Module

Contents

1	AI Assistance Module – LLM Selection & Technical Design	1
1.1	0. Positioning: Corti vs. the new AI module	1
1.2	1. Task 1 – LLM comparison and recommendation	2
1.2.1	Requirements derived from the meeting	2
1.2.2	Candidate comparison	2
1.2.3	Recommendation	3
1.3	2. Task 2 – Integration design	4
1.3.1	2.1 Where it lives (follows the existing architecture)	4
1.3.2	2.2 Backend interface (provider-agnostic by design)	5
1.3.3	2.3 Configuration (.env)	5
1.3.4	2.4 API contract	5
1.3.5	2.5 Grounding strategy (how “answer from this conversation only” is enforced) . . .	6
1.3.6	2.6 Frontend flow	6
1.3.7	2.7 Data flow (end-to-end)	7
1.3.8	2.8 Implementation phases	7
1.4	3. Anticipated questions & prepared answers	7

1 AI Assistance Module – LLM Selection & Technical Design

Deliverables for the next meeting: (1) LLM model comparison with a recommendation and justification, (2) integration design for the existing application, following the current code architecture.

1.1 0. Positioning: Corti vs. the new AI module

Decision: the AI Assistance module will NOT use the Corti API.

Concern	Corti (existing)	AI Assistance module (new)
Role	Speech-to-text, live clinical facts (FactsR(TM)), document templates	Post-recording summary, diagnosis extraction, transcript-grounded Q&A chat
Input	Audio (files / live streams)	Completed transcript text
Ownership	Third-party black box	Our own module – model is swappable and later fine-tunable on our transcription data

Rationale:

1. The project goal is to develop our own AI capability (“we will train our model on our transcription data”) – that requires a model we control, not a vendor API.
2. Corti’s integration surface in this app (STT streams, facts, document generation) has no transcript-grounded conversational Q&A endpoint. The Embedded Assistant’s chat is out of scope per project direction.
3. Separation of concerns matches the existing architecture: `internal/corti/` talks to Corti; the new `internal/ai/` talks to the LLM. Neither depends on the other – the transcript text is the only handoff between them.

1.2 1. Task 1 – LLM comparison and recommendation

1.2.1 Requirements derived from the meeting

- Prefer **small** models (no frontier-scale models needed)
- Prefer **open source**
- Prefer **free / near-zero cost**
- Tasks: summarization, diagnosis/allergy extraction, grounded Q&A over a 1-5 minute clinical conversation transcript (well within any modern model’s context window)
- Clinical transcripts are sensitive -> data privacy is a first-class criterion

1.2.2 Candidate comparison

Model	Size	Open source	Cost	Medical strength	Data privacy	Notes
Qwythos-9B (Claude-Mythos-5 distill, GGUF, local via Ollama) [recommended]	9B	yes Apache-2.0	\$0	Strong reasoning + instruction-following; explicit <think> reasoning traces	Transcript never leaves our server	Recommended primary – see highlights below
Llama 3.1 8B Instruct (local, via Ollama)	8B	yes	\$0	Good general clinical vocabulary; strong instruction-following	Local	Recommended alternate / fallback
Qwen 2.5 7B Instruct (local, via Ollama)	7B	yes	\$0	Comparable to Llama 8B; strong summarization	Local	Alternate – worth A/B testing
Mistral 7B Instruct (local, via Ollama)	7B	yes	\$0	Slightly weaker instruction-following than the two above	Local	Baseline option

Model	Size	Open source	Cost	Medical strength	Data privacy	Notes
Llama 3.3 70B via Groq API (free tier)	70B	yes (weights)	Free tier, rate-limited	Noticeably better reasoning/summaries	no Transcript sent to third party	Good demo fallback if local quality disappoints
Google Gemini Flash (free tier)	–	no	Free tier, rate-limited	Strong	no Third party	Closed model – conflicts with open-source preference
GPT-4o-mini / GPT-4.1-mini class APIs	–	no	~\$0.15-0.60 / 1M tokens	Strong	no Third party, paid	Conflicts with cost + open-source preference
BioMistral 7B / OpenBioLLM 8B / Meditron 7B (medical fine-tunes)	7-8B	yes	\$0	Medical-domain pretraining	Local	See “why not a medical model” below

1.2.3 Recommendation

Primary: Qwythos-9B-Claude-Mythos-5-1M (Empero AI), self-hosted via Ollama / llama.cpp, exposed through the OpenAI-compatible API.

1.2.3.1 Model highlights (from the published model card)

- **Base & method:** Qwen3.5-9B post-trained on 500M+ tokens of Claude Mythos/Fable chain-of-thought reasoning traces – a reasoning model that emits `<think>` blocks before answering, which improves multi-step extraction tasks like “list every diagnosis mentioned in this conversation”.
- **Publisher-reported benchmarks** (self-evaluated by Empero AI, not independently verified):
 - MMLU: **+34 points** over the Qwen3.5-9B base
 - GSM8K (strict): **+30 points**; GSM8K (flex): **+19 points**
 - Tool-use harness: **7/7** correct answers with source citations
- **1M-token context** (YaRN rope-scaling) – far beyond our needs for single consultations, but it means even very long clinic sessions or *batches* of transcripts fit in one prompt.
- **Native function calling** – useful for the later phase where the assistant might trigger structured actions (e.g., “add this to the document”).
- **Apache-2.0 license**, GGUF quantizations from Q4_K_M (5.63 GB – runs on an Apple-Silicon laptop) up to BF16.
- **Traction:** ~1.6M downloads/month and 1.6k+ likes on Hugging Face.

Why this one (the answer to bring to the meeting):

1. **Zero cost, local, private** – same as any Ollama-served model: no API keys, no billing, and clinical transcripts never leave our infrastructure.
2. **Reasoning quality per parameter** – the distilled chain-of-thought traces target exactly our workload (structured extraction + grounded Q&A), and the reported gains over its own base are large; at 9B it still runs on a development laptop.

3. **Open weights (Apache-2.0) & swappable** – served behind the same OpenAI-compatible endpoint, so falling back to Llama 3.1 8B or Qwen 2.5 7B is a one-line config change.
4. **Right-sized with headroom** – closed-context summarization/Q&A needs no frontier model; the 1M context and function calling give growth room for phases 4-5.
5. **Fine-tuning path** – Qwen-family base with standard architecture; LoRA/QLoRA tooling applies when we later train on our own transcription data.

Honest caveats (also our answers if the supervisor probes):

- The benchmark deltas are **self-reported by the publisher**; no independent leaderboard verification. We treat them as promising signals, and our own A/B test against Llama 3.1 8B on real sample transcripts (Phase 1) is the deciding evidence.
- It is a community fine-tune, not an official Qwen/Meta release; the card itself warns the model “can over-commit to specific identifiers it isn’t certain about” – which is precisely why our design grounds every answer in the transcript (§2.5) and uses moderate temperature (the card advises against greedy sampling, which can loop).
- The <think> reasoning blocks must be stripped server-side before responses reach the UI (handled in internal/ai, §2.5).

Why **not** a medical fine-tune (Meditron / BioMistral / OpenBioLLM) as the default: they are research models tuned for medical *knowledge benchmarks* (exam-style QA), not for instruction-following, summarization, or grounded chat. For “summarize this conversation and answer questions only from it”, a strong general instruct model outperforms them in practice. They remain candidates for the future fine-tuning phase.

Why not a hosted API as the default: cost, rate limits, closed weights (Gemini/GPT), and PHI leaving our environment. Groq’s free tier (open Llama weights, hosted) is kept as a documented fallback if local inference quality or speed is insufficient for the demo.

1.3 2. Task 2 – Integration design

1.3.1 2.1 Where it lives (follows the existing architecture)

```
backend/
`-- internal/
    |-- ai/                                <- NEW: all LLM-related code, nothing Corti-related
    |   |-- client.go                      <- LLMClient interface (provider-agnostic)
    |   |-- openai_compat.go              <- Implementation for any OpenAI-compatible endpoint
    |   |                                  (Ollama local, Groq, etc. -- selected by config)
    |   |-- prompts.go                    <- System prompts: summarizer, extractor, grounded chat
    |   |-- types.go                      <- Request/response structs
    |-- corti/                            <- unchanged
    |-- handlers/
    |   |-- ai_handler.go                 <- NEW: HTTP handlers (mirrors ambient_handler.go style)
    |-- middleware/                      <- unchanged
    |-- utils/env.go                     <- + AI_* config keys

frontend/src/
|-- composables/
|   |-- useAiAssistant.ts                <- NEW: API calls + chat state (mirrors useAmbientSession)
|-- components/
|   |-- AiAssistantPanel.vue             <- NEW: summary card + chat UI
`-- views/AmbientSessionPage.vue         <- MODIFIED: "AI Assistance" button + panel at the
                                         bottom of the live-transcript section
```

1.3.2 2.2 Backend interface (provider-agnostic by design)

```
// internal/ai/client.go
type LLMClient interface {
    Summarize(ctx context.Context, transcript string) (*SummaryResult, error)
    Chat(ctx context.Context, transcript string, history []ChatMessage, question string) (string, error)
}

type SummaryResult struct {
    Summary      string `json:"summary"`
    Diagnoses    []string `json:"diagnoses"`
    Allergies    []string `json:"allergies"`
    KeyPoints    []string `json:"key_points"`
}
```

One implementation (`openai_compat.go`) covers every provider we care about, because Ollama, Groq, and OpenAI all speak the same `/v1/chat/completions` protocol. The provider is chosen purely by configuration.

1.3.3 2.3 Configuration (.env)

```
# AI Assistance module
AI_BASE_URL=http://localhost:11434/v1      # Ollama default; point at Groq/other to swap
# Ollama can pull GGUF models straight from Hugging Face:
#   ollama pull hf.co/empero-ai/Qwythos-9B-Claude-Mythos-5-1M-GGUF:Q4_K_M
AI_MODEL=hf.co/empero-ai/Qwythos-9B-Claude-Mythos-5-1M-GGUF:Q4_K_M
# Fallback: AI_MODEL=llama3.1:8b
AI_API_KEY=                               # empty for local Ollama; required for hosted
AI_TIMEOUT_SECONDS=120
```

1.3.4 2.4 API contract

Both endpoints are **stateless** (the transcript travels with each request – consistent with the app’s existing patterns, no new storage needed; a 5-minute conversation is only a few thousand tokens).

Routes registered in `main.go`, protected by the standard middleware chain, gated by `ambient.access` initially (optionally a dedicated `ai_assistant.access` permission added to `internal/auth/permissions.go`, following the RBAC v2 pattern):

1.3.4.1 POST /api/ai/summarize Called once when the user clicks **AI Assistance** after a recording completes.

```
// Request
{
    "transcript": "Doctor: Good morning... Patient: I've had severe headaches...",
    "language": "en" // optional, defaults to "en"
}

// Response 200
{
    "success": true,
    "summary": "Patient reports severe headaches for the past week with stroke-like symptoms...",
    "diagnoses": ["Suspected neurological condition -- specialist referral advised"],
    "allergies": [],
    "key_points": ["Severe headache, 1 week duration", "Stroke-like symptoms", "Referred to specialist"]
}
```

```

}

// Response 4xx/5xx
{ "success": false, "error": "AI service unavailable", "message": "..." }

```

1.3.4.2 POST /api/ai/chat Called for each follow-up question. history carries prior Q&A turns so the conversation is coherent; answers must be grounded in the transcript only.

```

// Request
{
  "transcript": "...full transcript...",
  "history": [
    { "role": "user", "content": "What are the diagnoses?" },
    { "role": "assistant", "content": "The doctor suspects..." }
  ],
  "question": "What allergies were mentioned?"
}

// Response 200
{ "success": true, "answer": "No allergies were mentioned in this conversation." }

```

1.3.5 2.5 Grounding strategy (how “answer from this conversation only” is enforced)

- System prompt pins the model to the transcript: *“You are a clinical documentation assistant. Answer ONLY from the transcript provided between the delimiters. If the transcript does not contain the answer, say so explicitly. Do not use outside medical knowledge to invent facts. This is documentation support, not medical advice.”*
- Transcript is injected inside clear delimiters in the system message; user questions never mix with it.
- Summarize requests structured JSON output (model is asked for a JSON object; backend validates/parses, with a plain-text fallback).
- **Reasoning-trace handling:** Qwythos emits <think>...</think> blocks before its answer. internal/ai strips these server-side so only the final answer reaches the frontend (the traces can be logged at debug level for prompt tuning).
- **Sampling:** temperature ~0.6-0.7 per the model card’s guidance (greedy/very-low temperature can cause repetition loops with this model).
- A one-line disclaimer accompanies AI output in the UI (documentation aid, verify clinically).

1.3.6 2.6 Frontend flow

```

AmbientSessionPage (recording finished)
  | [AI Assistance] button enabled (bottom of live-transcript section)
  v click
useAiAssistant.summarize(getFullTranscript())
  | POST /api/ai/summarize
  v
AiAssistantPanel renders: Summary * Diagnoses * Allergies * Key points
  | user types question
  v
useAiAssistant.ask(question)      // POST /api/ai/chat with history
  v
Answer appended to chat thread (grounded in transcript only)

```

The panel reuses the app’s existing conventions: `getAuthHeaders()` / `handleFetchResponse()` from the auth store, Vuetify cards matching the Clinical Facts styling, loading/error states like `useTranscription`.

1.3.7 2.7 Data flow (end-to-end)

```

Microphone --> Browser (MediaRecorder, 500ms chunks)
                | WS /api/ambient/ws/:id
                v
Go backend (AmbientProxy) <----- OAuth2 token (TokenManager)
                | WS (proxied)
                v
Corti API -- transcripts + facts --> browser (live view)
                | recording ends
                v
                full transcript (frontend state)
                | POST /api/ai/summarize | /api/ai/chat
                v
Go backend (internal/ai -> handlers/ai_handler)
                | /v1/chat/completions
                v
Ollama (Llama 3.1 8B, local -- no data egress)

```

1.3.8 2.8 Implementation phases

Phase	Scope	Outcome
1	Ollama setup + internal/ai client + /api/ai/summarize + minimal panel; A/B Qwythos-9B vs Llama 3.1 8B on sample transcripts	Summary + diagnoses render after a recording; model choice validated on our own data
2	/api/ai/chat + chat UI with history	Grounded Q&A working
3	Polish: structured-output validation, error/timeout handling, optional RBAC permission, save AI results with the session	Demo-ready
4 (later)	Same panel on the File Transcription tab	Per supervisor's roadmap
5 (future)	Fine-tune on collected transcription data; swap model via config	Custom model milestone

1.4 3. Anticipated questions & prepared answers

- **Why Qwythos-9B?** Free, open (Apache-2.0), private (local), reasoning-distilled for exactly our workload (structured extraction + grounded Q&A), 1M context and function calling for later phases, and still laptop-sized at 9B/Q4.
- **Are the benchmarks trustworthy?** They are publisher-reported, not independently verified – we say so openly. Our Phase-1 A/B test against Llama 3.1 8B on real sample transcripts is the evidence we will actually rely on; the architecture makes the swap a config change either way.
- **Why not a medical LLM?** Medical fine-tunes optimize exam-style knowledge recall, not instruction-following or grounded summarization; a strong general instruct model does this task better. They stay on the roadmap as fine-tuning bases.

- **Why not GPT/Gemini APIs?** Cost, rate limits, closed weights, and clinical transcripts leaving our environment.
- **Do I need to pay for APIs?** No – the recommended setup costs \$0 (local Ollama). Groq free tier is the documented fallback.
- **How do we swap models later?** One env var (`AI_MODEL` / `AI_BASE_URL`); the `LLMClient` interface and OpenAI-compatible protocol make the code provider-agnostic.